ESPCI PARIS | PSL

Résolution numérique de problèmes de physique quantique

Maxime Rimbaud

144

Table des matières

1 L'oscillateur harmonique dans tous ses états (quantiques) 3
1.1 Étude préliminaire . . . 3
1.1.1 Puits de potentiel infini . . . 3
1.1.2 Potentiel harmonique dans un puits infini . . . 4
1.2 Potentiel normalisé et solutions . . . 4
1.3 Résolution numérique . . . 5
1.3.1 symétrie de la matrcie **H** . . . 5
1.3.2 Energies propres . . . 7
1.3.3 Fonctions propres . . . 9

2 Puits de potentiel fini dans les hétérostructures de semi-conducteurs 11
2.1 Modèle du puits de potentiel . . . 11
2.2 Résolution numérique . . . 11
2.2.1 Base de fonctions propres . . . 11
2.2.2 Solutions numériques . . . 11
2.2.3 Lien avec les solutions graphiques du TD n°1 . . . 12

3 Conclusion 13

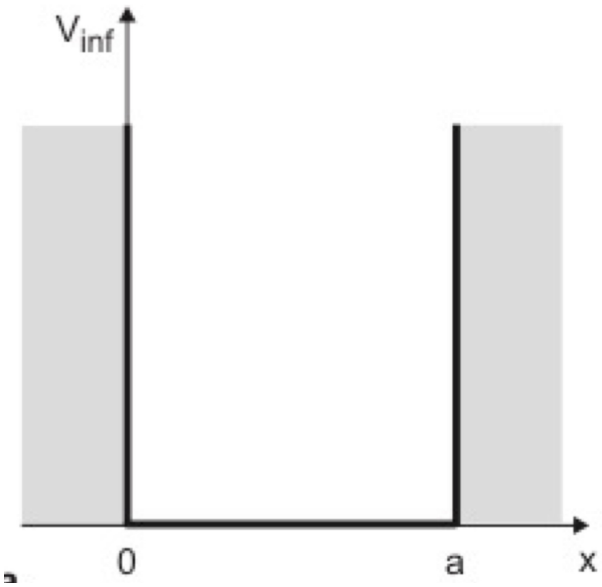
4 Annexes 13
4.1 Scripts Partie I . . . 13
4.2 Scripts Partie II . . . 16

1 L'oscillateur harmonique dans tous ses états (quantiques)

1.1 Étude préliminaire

1.1.1 Puits de potentiel infini

Considérons une particule de masse m confinée dans le puits de potentiel infini ci-dessous :



L'hamiltonien du système en représentation |x> est rappelé dans le sujet :

$$H_0 = \frac{-\hbar^2}{2.m}\frac{d^2}{dx^2} + V_{inf}(x)$$

Avec $V_{inf}(x) = 0$ si $0 < x < a$ et $V_{inf}(x) = +\infty$ sinon
On cherche donc à résoudre l'équation aux valeurs propres suivante :

$$H_0\varphi = E\varphi$$

Condition aux limites : la fonction d'onde doit être continue donc $\varphi(x = 0) = 0$ et $\varphi(x = a) = 0$
On se place dans le cas où $E > 0$ car si $E \leq 0$ il n'y a pas de quanton possible. Dans le puits en représentation |x>, on a :

$$\frac{d^2\varphi}{dx^2} + \frac{2.m.E}{\hbar^2}\varphi = 0$$

Posons $k = \frac{\sqrt{2.m.E}}{\hbar}$, on a alors :

$$\varphi(x) = A.\cos(k.x) + B.\sin(k.x)$$

D'après les conditions aux limites :

$$\varphi(0) = A = 0$$

$$\varphi(a) = B.sin(k.a) = 0 \Rightarrow k_n.a = n.\pi \Rightarrow k_n = \frac{n.\pi}{a}$$

Ainsi :

$$E_n = \frac{n^2.\pi^2.\hbar^2}{2.m.a^2}$$

Déterminons B :

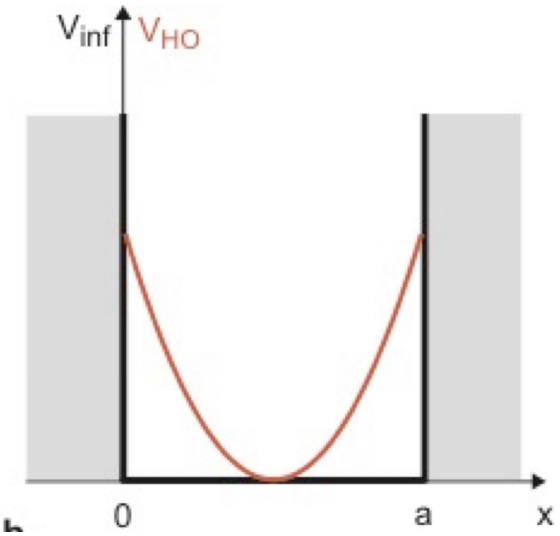
$$\int_0^a |\varphi_n(x)|^2.dx = 1 \Rightarrow B^2.\int_0^a \sin(k.x)^2.dx = 1 \Rightarrow B.\sqrt{\frac{a}{2}} = 1 \Rightarrow B = \sqrt{\frac{2}{a}}$$

Finalement :

$$\varphi_n(x) = \sqrt{\frac{2}{a}}.\sin(\frac{n.\pi.x}{a})$$

1.1.2 Potentiel harmonique dans un puits infini

Considérons maintenant le cas d'un potentiel harmonique localisé à l'intérieur du puits infini :



L'hamiltonien du système en représentation |x> est rappelé dans le sujet :

$$H = H_0 + V_{OH}$$

où $V_{OH} = \frac{1}{2}.m.\omega^2(x - \frac{a}{2})^2$

Posons $|\Psi >= \sum_{m=1}^{+\infty} c_m.|\varphi_m >$, en réalisant le produit scalaire entre l'équation de Schrödinger $H|\Psi >= E|\Psi >$ avec un bra $< \varphi_n|$, on a :

$$< \varphi_n|H|\Psi >= E < \varphi_n|\Psi >= E.c_n$$

Or en exploitant la représentation matricielle de l'hamiltonien, on a :

$$H|\Psi >= \sum_{k=1}^{+\infty}\sum_{m=1}^{+\infty} H_{k,m}c_m|\varphi_k >$$

$$\Rightarrow < \varphi_n|H|\Psi >= \sum_{k=1}^{+\infty}\sum_{m=1}^{+\infty} H_{k,m}c_m < \varphi_n|\varphi_k >= \sum_{m=1}^{+\infty} H_{n,m}c_m$$

D'où l'équation suivante valable pour tout entier naturel n non nul :

$$\sum_{m=1}^{+\infty} H_{n,m}c_m = E.c_n$$

1.2 Potentiel normalisé et solutions

a. Exprimons le potentiel harmonique sous forme normalisée :

$$\nu_{OH} = \frac{V_{OH}}{E_1^{(0)}} = \frac{1}{(E_1^{(0)})^2}\frac{1}{2}.m.\omega^2(x - \frac{a}{2})^2.\frac{\pi^2.\hbar^2}{2.m.a^2} = \frac{\pi^2}{4}.(\frac{\hbar.\omega}{E_1^{(0)}})^2.(\frac{x}{a} - \frac{1}{2})^2$$

b. Exprimons les coefficients de **H** normalisés :

$$h_{n,m} = \frac{H_{n,m}}{E_1^{(0)}} = n^2.\delta_{n,m} + \frac{2}{a}.\int_0^1 \sin(\frac{n.\pi.x}{a}).\frac{V_{OH}}{E_1^{(0)}}.\sin(\frac{m.\pi.x}{a}).dx$$

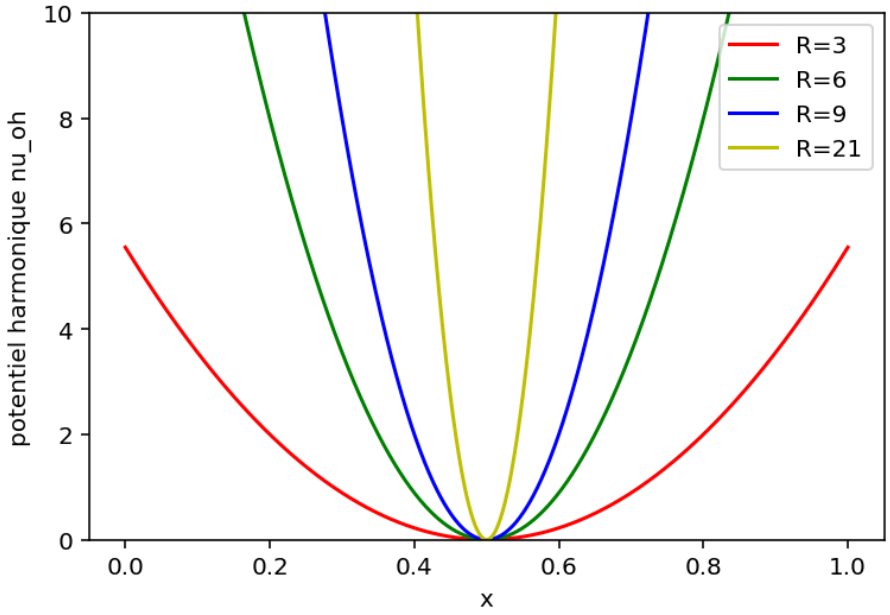
Soit en posant $x' = \frac{x}{a}$ avec $dx' = \frac{dx}{a}$:

$$h_{n,m} = n^2.\delta_{n,m} + 2.\int_0^1 \sin(n.\pi.x').\nu_{OH}.\sin(m.\pi.x').dx'$$

c. Exprimons ν_{OH} en fonction de R :

$$\nu_{OH} = \frac{\pi^2}{4}.R^2.(\frac{x}{a} - \frac{1}{2})^2$$

On peut alors tracer le potentiel harmonique pour différentes valeurs de R :



1.3 Résolution numérique

1.3.1 symétrie de la matrcie H

J'ai arbitrairement choisi d'afficher **H** avec des coefficients précis à 10^{-3} pour simplifier la lecture :

$$\mathbf{H} = \begin{bmatrix} 73.555 & 0.000 & 84.375 & 0.000 & 15.625 & 0.000 \\ 0.000 & 160.930 & 0.000 & 100.000 & 0.000 & 21.094 \\ 84.375 & 0.000 & 181.555 & 0.000 & 105.469 & 0.000 \\ 0.000 & 100.000 & 0.000 & 194.024 & 0.000 & 108.000 \\ 15.625 & 0.000 & 105.469 & 0.000 & 205.555 & 0.000 \\ 0.000 & 21.094 & 0.000 & 108.000 & 0.000 & 217.930 \end{bmatrix}$$

On constate que **H** est symétrique et que si n et m ont la même parité $H_{n,m} = 0$, montrons cela :

$$h_{n,m} = n^2.\delta_{n,m} + 2.\int_0^1 \sin(n.\pi.x).\nu_{OH}(x).\sin(m.\pi.x).dx = n^2.\delta_{n,m} + \int_0^1 \nu_{OH}(x).(\cos((n-m).\pi.x) - \cos((n+m).\pi.x)).dx$$

Si $n \neq m$, on a :

$$h_{n,m} = \int_0^1 \nu_{OH}(x).(\cos((n-m).\pi.x) - \cos((n+m).\pi.x)).dx = \int_0^1 \nu_{OH}(x).(\cos((m-n).\pi.x) - \cos((n+m).\pi.x)).dx = h_{m,n}$$

Grâce à la parité de la fonction cosinus on a bien **H** symétrique.

$$h_{n,m} = \frac{\pi^2.R^2}{4}\int_0^1 (x - \frac{1}{2})^2.(\cos((n-m).\pi.x) - \cos((n+m).\pi.x)).dx$$

Posons maintenant (pour $k \neq 0$) :

$$I_k = \frac{\pi^2.R^2}{4} \int_0^1 (x - \frac{1}{2})^2 . \cos(k.\pi.x).dx$$

En réalisant deux intégrations par parties successives on a :

$$\begin{aligned}
I_k &= \frac{\pi^2 R^2}{4} \int_0^1 \left(x - \frac{1}{2}\right)^2 \cos(k\pi x)\, dx \\
&= \frac{\pi^2 R^2}{4} \left[\left[\left(x - \frac{1}{2}\right)^2 \frac{\sin(k\pi x)}{k\pi} \right]_0^1 - \int_0^1 2 \left(x - \frac{1}{2}\right) \frac{\sin(k\pi x)}{k\pi}\, dx \right] \\
&= \frac{\pi^2 R^2}{4} \left(-2 \int_0^1 \left(x - \frac{1}{2}\right) \frac{\sin(k\pi x)}{k\pi}\, dx \right) \\
&= \frac{\pi^2 R^2}{4} \left(2 \left[\left(x - \frac{1}{2}\right) \frac{\cos(k\pi x)}{(k\pi)^2} \right]_0^1 - \int_0^1 \frac{\cos(k\pi x)}{(k\pi)^2}\, dx \right) \\
&= \frac{\pi^2 R^2}{2} \left[\frac{1}{2} \frac{1}{(k\pi)^2} (-1)^k + \frac{1}{2} \frac{1}{(k\pi)^2} \right] \\
&= \frac{R^2}{4.k^2} \left(1 + (-1)^k\right).
\end{aligned}$$

On en déduit que $\forall n, m \ / \ n \neq m$:

$$h_{n,m} = I_{n-m} - I_{n+m} = \begin{cases} 0 & \text{si } n \text{ et } m \text{ ne sont pas de même parité,} \\ \dfrac{R^2}{2} \dfrac{4nm}{(n^2 - m^2)^2} & \text{sinon.} \end{cases}$$

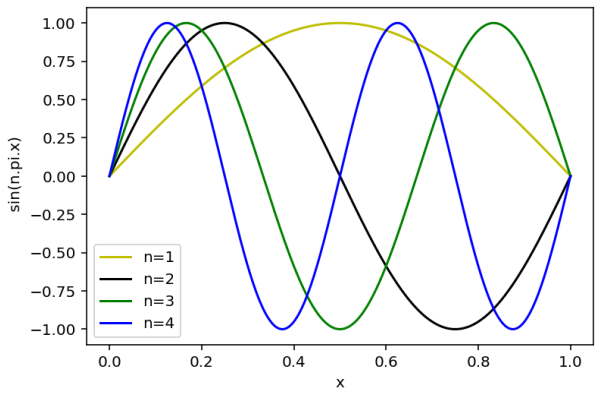
et si $n = m$:

$$h_{n,n} = n^2 + \frac{\pi^2.R^2}{4} \int_0^1 (x - \frac{1}{2})^2 .dx - I_{2n} = n^2 + \frac{\pi^2.R^2}{4} \frac{1}{12} - \frac{R^2}{8.n^2}$$

Soit

$$h_{n,n} = n^2 + \frac{R^2}{8} (\frac{\pi^2}{6} - \frac{1}{n^2})$$

Interprétation physique : $h_{n,m}$ représente le couplage énergétique entre le vecteur $|\varphi_n>$ et le vecteur $|\varphi_m>$. Lorsque ce terme est nul on a une énergie d'intéraction nulle entre ces deux états. Or $\varphi_n(x)$ est propotionnelle à $sin(nx)$ et cette fonction est paire (respectivement impaire) par rapport à l'axe verticale passant par x=0.5, si n est impaire (respectivement paire). On le constate sur le graphique suivant :

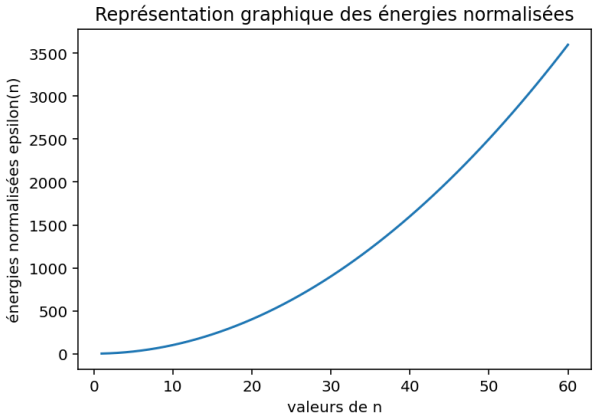


Ainsi les fonctions paires et impaires par rapport à l'axe verticale passant par x=0.5 sont découplées, on peut donc distinguer deux classes de vecteurs.

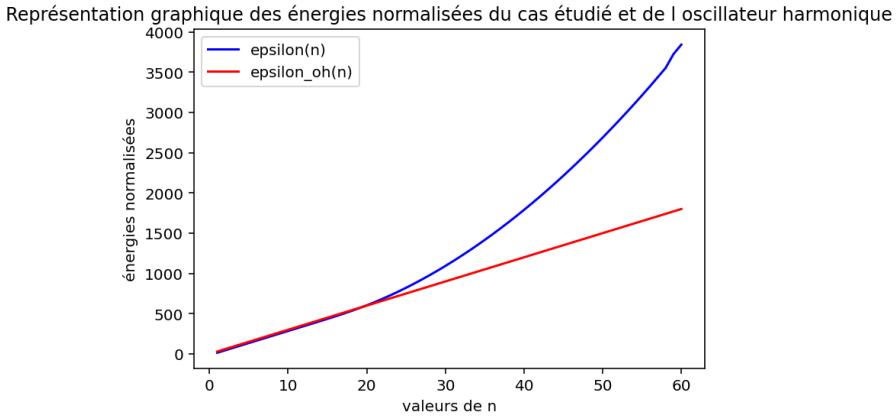
1.3.2 Energies propres

N est fixé à 60 et nous allons déterminer numériquement les énergies propres normalisées associées au système : $\epsilon(n) = E(n)/E_1^0$. Comme il est rappelé dans l'énoncé, l'objectif de cette partie est de déterminer les énergies propres d'une particule soumise à un potenitel harmonique tronqué par un puits infini. On comprend donc le sens de la comparaison des $\epsilon(n)$ avec les énergies normalisées théoriques d'un potentiel harmonique.

a. En calculant numériquement les valeurs de $\epsilon(n)$ et en les traçant en fonction de n, on obtient :



b. On normalise E_{HO} par E_1^0 pour comparer des choses comparables et en traçant $\epsilon(n)$ et $\epsilon_{OH}(n)$ sur la même figure, on obtient :

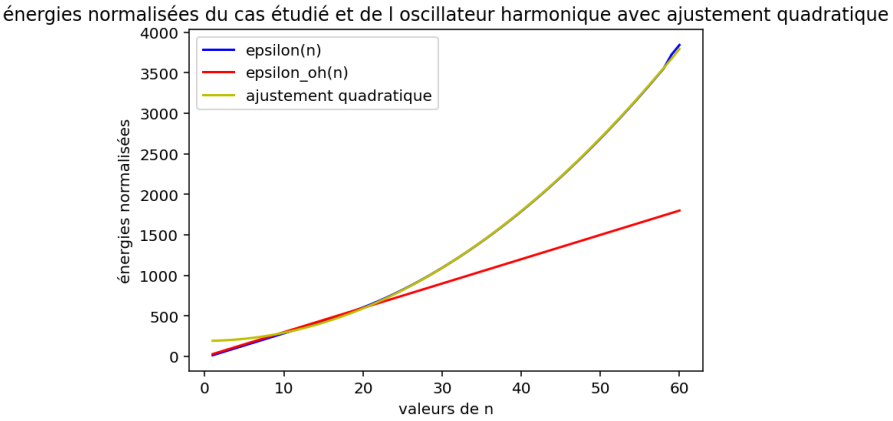


On constate une très bonne correspondance pour les valeurs de n inférieures à 20.

c.On constate également que pour n grand $\epsilon(n)$ change d'allure et devient quadratique et on souhaite montrer que les énergies propres sont alors bien ajustées par une courbe de la forme $f(n) = n^2 + C$. On peut dans un premier temps déterminer C en minimisant l'erreur quadratique :

$$C = \frac{1}{N} \sum_{n=1}^{N} \epsilon(n) - n^2$$

Voir la section minimisation de l'erreur quadratique dans le code python en annexe.
On obtient alors la représentation graphique suivante :



La valeur de C determinée est : $C = 192,611$

Mais d'où provient C ?
Rappellons que l'équation (8) nous a permis d'exprimer littéralement le coefficient $h_{n,n}$:

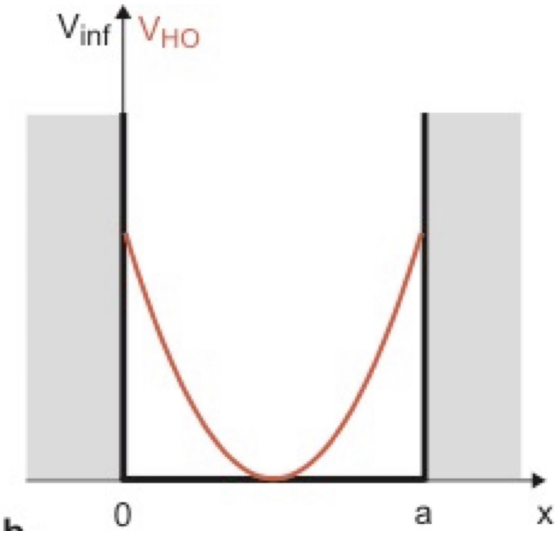
$$h_{n,n} = n^2 + \frac{R^2}{8}(\frac{\pi^2}{6} - \frac{1}{n^2})$$

Or pour n très grand le terme en $\frac{1}{n^2}$ est négligeable et on a une expression des coefficients diagonaux :

$$h_{n,n} \underset{n\to+\infty}{\approx} n^2 + \frac{\pi^2.R^2}{48}$$

De plus les coefficients diagonaux sont très grands en valeurs absolues devant les coefficients non diagonaux, on peut donc considérer que les valeurs propres on le même comportement que les coefficients diagonaux. On a donc trouvé l'origine de C : $C(R) \approx \frac{\pi^2.R^2}{48}$.

d. On peut visualiser la transition entre les deux régimes à l'aide de la figure ci-dessous :



Lorsque l'énergie normalisée de la particule va dépasser la valeur du potentiel normalisé de l'oscillateur harmonique celle-ci sera contrainte par les parois du puits infini. Le poteniel harmonique est alors négligeable devant le potentiel du puits infini et on observe la transition de régime.
Notons que $\nu_{OH}(1) = \frac{\pi^2.R^2}{16} = 555,165$ qui correspond donc à l'énergie limite de la particule à partir de laquelle on observe la transition de régime. En comparant cette valeur aux valeurs $\epsilon(n)$, on trouve $n_0 = 18$ avec $\epsilon(n_0) = 566,873$

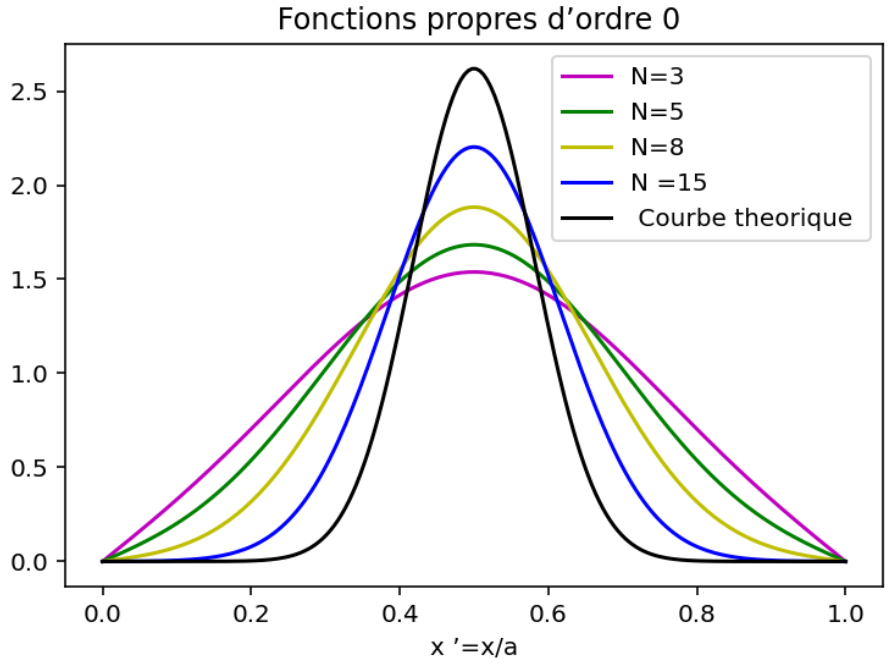
1.3.3 Fonctions propres

On prend maintenant a=1 et on va étudier les fonctions propres de la particule.

a. Notons qu'on peut exprimer les fonctions propres dans la base constituée par les fonctions φ_n définies dans le préliminaire :

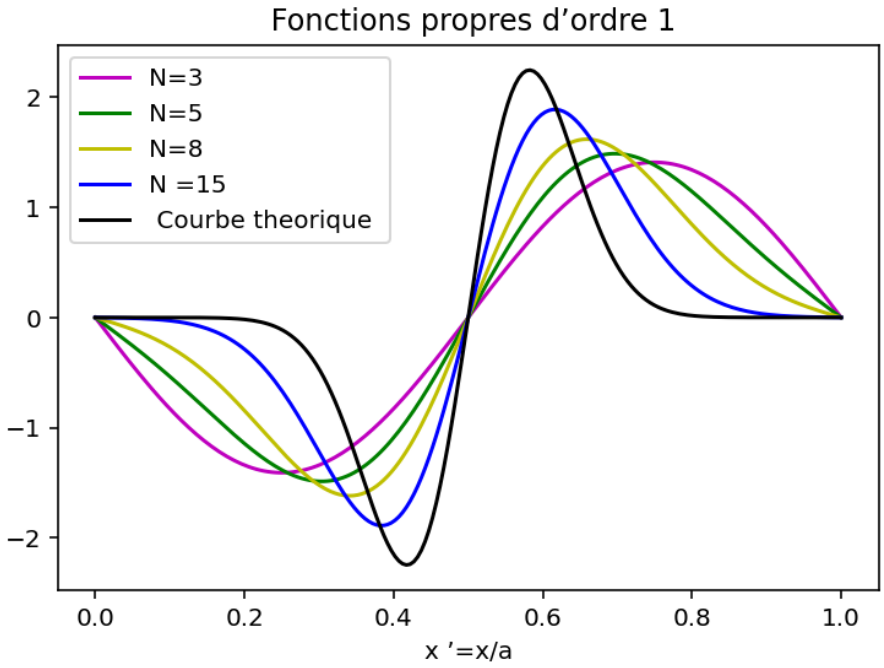
$$\Psi_n(x) = \sum_i c_i^n . \varphi_n(x)$$

Où les coefficients c_i^n correspondent à la i-ème coordonnée de Ψ_n dans la base φ_n.
On peut donc déterminer numériquement la fonction d'onde de l'état fondamental $\Psi_0(x)$ pour différentes valeurs de N et lorsqu'on les trace avec la fonction théorique de l'énoncé on obtient :



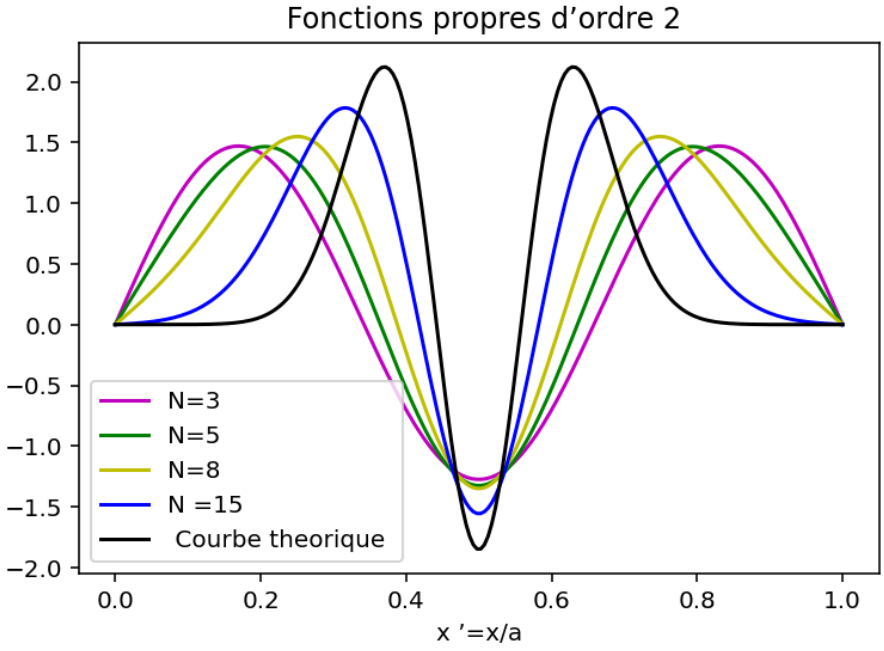
On remarque que plus N est grand plus la correspondance avec la fonction théorique est bonne.

b. En faisant la même chose avec l'état $\Psi_1(x)$, on obtient :



À nouveau on remarque que plus N est grand plus le modèle théorique est pertinent.

c. Pour l'état $\Psi_2(x)$, on obtient :

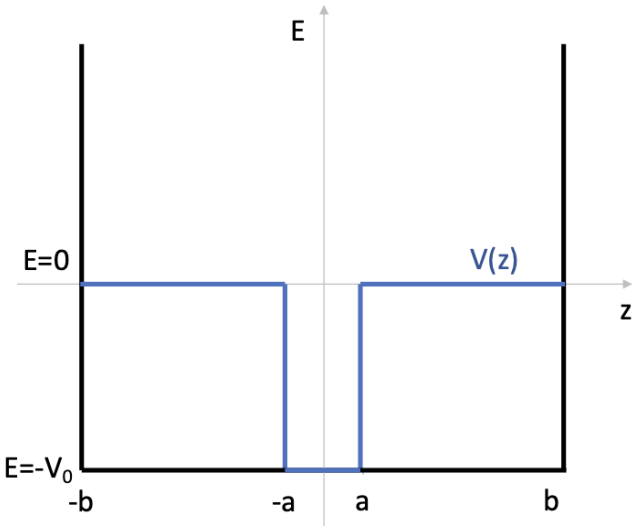


Et on fait à nouveau le constat que plus N est grand plus nos courbes s'approchent de la représentation théorique.

2 Puits de potentiel fini dans les hétérostructures de semi-conducteurs

2.1 Modèle du puits de potentiel

On considère la situation présentée sur la figure ci-dessous où le puits de potentiel fini qui décrit la structure $Al_xGa_{1-x}As/GaAs/Al_xGa_{1-x}As$ a une largeur $2a$ et une profondeur $-V_0$. Ce puits de potentiel est tronqué par un puits infini de largeur $2b$ et de profondeur $-V_0$.



L'hamiltonien du système est :

$$H = H_0 + V(z)$$

où $H_0 = \frac{p^2}{2m} - V_0$ correspond à l'hamiltonien du puits infini. On notera la présence du potentiel $-V_0$ qui implique que les énergies des modes propres sont décalées de $-V_0$ par rapport aux solutions du puits infini vu en cours et où $V(z) = 0$ si $|z| < a$ et $V(z) = V_0$ si $b > |z| > a$. La masse m est la masse d'un électron et on prendra les paramètres suivants : a=1nm et $V_0 = 0.5$eV. Le paramètre b sera choisi suffisamment grand pour assurer la décroissance des fonctions d'onde en dehors du puits fini, par exemple b=5a.

2.2 Résolution numérique

2.2.1 Base de fonctions propres

On obtient les fonctions propres du cas présentés ci-dessus en reprenant les fonctions trouvées au I.A) et en posant $a = 2b$ et $x = z + b$ de plus les énergies propres sont simplement abaissées de $-V_0$ d'où :

$$\varphi_n(z) = \sqrt{\frac{1}{b}} \sin(\frac{n.\pi}{2b}(z+b))$$

$$E_n = \frac{n^2.\hbar^2.\pi^2}{8.m.b^2} - V_0$$

2.2.2 Solutions numériques

Pour déterminer les états confinés du puits de potentiel fini, nous allons appliquer la même méthode numérique que dans la partie I, en développant la fonction d'onde sur la base des états propres du puits infini tronqué de largeur $2b$. Les fonctions propres utilisées sont les φ_n déterminées ci-dessus tout comme les énergies correspondantes

La matrice de l'Hamiltonien dans cette base s'écrit alors :

$$H_{nm} = E_n^{(0)}\,\delta_{nm} + \int_{-b}^{b} \varphi_n(z)\,V(z)\,\varphi_m(z)\,dz,$$

où

$$V(z) = \begin{cases} 0 & \text{si } |z| < a, \\ V_0 & \text{si } a < |z| < b. \end{cases}$$

La matrice H a été construite numériquement avec une base de taille $N = 150$. La diagonalisation de H fournit l'ensemble des valeurs propres.
Après avoir trié les valeurs propres de H, j'ai affiché les 5 plus basses énergies :

$$E_0 = -0.44365995, \qquad E_1 = -0.2814861, \qquad E_2 = -0.05117982, \qquad E_3 = 0.02108887, \qquad E_4 = 0.02783029,$$

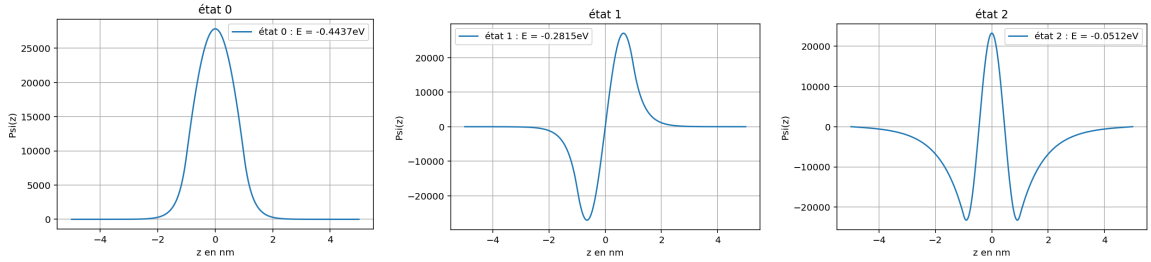
Les énergies négatives correspondent aux états confinés dans le puits fini, et nous trouvons trois états confinés, en accord avec le résultat attendu :

$$E_0 = -0.44365995, \qquad E_1 = -0.2814861, \qquad E_2 = -0.05117982,$$

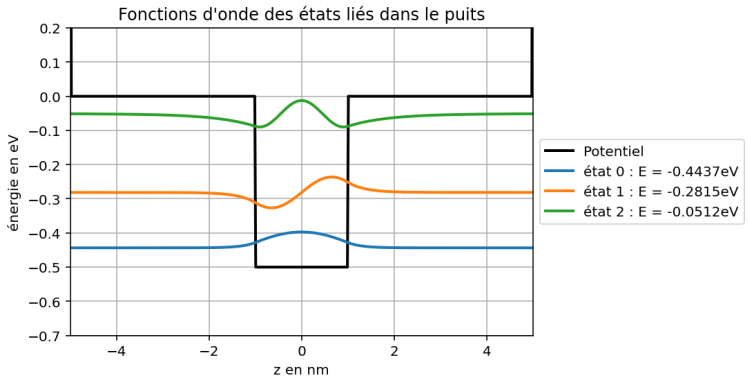
Les fonctions d'onde associées sont reconstruites via :

$$\Psi_n(z) = \sum_{k=1}^{N} c_k^{(n)} \, \varphi_k(z),$$

Après les avoir normalisées, on peut les tracer :



Leur tracé superposé au potentiel $V(z)$ donne :



On retrouve la structure donnée pour un puits fini dans le sujet.

2.2.3 Lien avec les solutions graphiques du TD n°1

Dans le TD n°1, nous avions montré que les niveaux d'énergie d'un puits fini sont obtenus à partir des équations suivantes :

$$|\cos(q_n a)| = \frac{q_n}{q} \qquad \text{(états pairs)},$$

$$|\sin(q_n a)| = \frac{q_n}{q} \qquad \text{(états impairs)},$$

où

$$V_0 = \frac{\hbar^2 q^2}{2m}, \qquad E_n = -\frac{\hbar^2 k_n^2}{2m}.$$

Avec $q^2 = q_n^2 + k^2$
On peut alors exprimer k_n, et q :

$$k_n = \sqrt{\frac{-2mE_n}{\hbar^2}}, \qquad q = \sqrt{\frac{2mV_0}{\hbar^2}} = 3,66.10^9 \mathrm{m}^{-1}$$

La représentation des fonctions d'onde des états liés dans le puits montre que : l'état 0 est paire, l'état 1 est impaire et l'état 2 est paire.
Calculons maintenant les différentes valeurs de k_n et q_n :

Pour l'état 0 :

$$k_0 = 3,45.10^9 \mathrm{m}^{-1}, \qquad q_0 = 1,22.10^9 \mathrm{m}^{-1}$$

D'où : $|\cos q_0 a| = 0,34$ et $\frac{q_0}{q} = 0,33$

Pour l'état 1 :

$$k_1 = 2,74.10^9 \mathrm{m}^{-1}, \qquad q_1 = 2,42.10^9 \mathrm{m}^{-1}$$

D'où : $|\sin q_1 a| = 0,66$ et $\frac{q_1}{q} = 0,66$

Pour l'état 2 :

$$k_2 = 1,17.10^9 \mathrm{m}^{-1}, \qquad q_2 = 3,47.10^9 \mathrm{m}^{-1}$$

D'où : $|\cos q_2 a| = 0,95$ et $\frac{q_2}{q} = 0,95$

Cette excellente concordance démontre que :

1. la méthode numérique matricielle reproduit fidèlement les conditions de quantification du puits de potentiel fini,
2. les trois niveaux négatifs trouvés correspondent bien aux seules solutions physiquement confinées dans le puits GaAs.

Ainsi, les solutions numériques et les solutions graphiques du TD sont parfaitement cohérentes (centième près).

3 Conclusion

Ce travail nous a donc permis d'obtenir des solutions numériques approchées pour l'oscillateur harmonique quantique et de les comparer aux solutions analytiques établies en cours, validant ainsi de manière rigoureuse nos résultats. Nous avons mis en évidence la signification des états confinés et leur représentation, notions qui restent difficiles à appréhender sans visualisation directe.

4 Annexes

4.1 Scripts Partie I

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Sat Nov 15 14:15:33 2025

```

```
@author: maximerimbaud
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad
from pprint import pprint

m = 9.1*10**(-31)                    # en kg
hbar = 1.05*10**(-34)                # en J/rad/s
a = 10**(-9)                         # en m
E_1 = np.pi**2*hbar**2/(2*m*a**2)

# Question I.A.3.c
X = np.linspace(0,1,1000)
def Voh(R,x):
    return (m/(2*hbar**2))*R**2*E_1**2*(x-a/2)**2
def nu_oh(R,x):
    return ((np.pi**2)/4)*(R**2)*(x-1/2)**2

plt.figure(1)
plt.plot(X,nu_oh(3,X),'r', label = "R=3")
plt.plot(X,nu_oh(6,X),'g', label = "R=6")
plt.plot(X,nu_oh(9,X),'b', label = "R=9")
plt.plot(X,nu_oh(21,X),'y', label = "R=21")
plt.ylim ([0 ,10])
plt.xlabel ('x')
plt.ylabel ('potentiel harmonique nu_oh')
plt.legend ()
plt.show()



# Question I.B.1

def delta(n,m):
    if n==m:
        return 1
    return 0



def f(x,n,m,R):                                                    # fonction    int grer pour exprimer les coefficients de H
    return np.sin(n*np.pi*x)*np.sin(m*np.pi*x)*nu_oh(R,x)

def H(R,N):
    MH=[[0 for i in range (0,N)] for i in range (0,N)]             # on cr e une matrice nulle avec les bonnes dimensions
    for n in range (0,N):
        for m in range (0,N):
            MH[n][m]= format(delta(n+1,m+1)*(n+1)**2 + 2*quad(lambda x:
                f(x,n+1,m+1,R),0,1)[0],'3f') # on calcule chaque coefficients de la matrice
    return np.array(MH,dtype=float)

## interpr tation physique
def sinus(n,x):
    return np.sin(n*np.pi*x)
x = np.linspace(0, 1,1000)

plt.figure(2)
plt.plot (x, sinus(1,x) ,'y', label ='n=1')
plt.plot (x, sinus(2,x) ,'k', label ='n=2')
plt.plot (x, sinus(3,x) ,'g', label ='n=3')
plt.plot (x, sinus(4,x) ,'b', label ='n=4')
plt.legend ()
plt.ylabel ('sin(n.pi.x)')
plt.xlabel ('x')
```

```
plt.show()

# Question I.B.2.a

N= 60
valeurs_propres = np.linalg.eigvals(H(30,60))
valeurs_propres_H = valeurs_propres.tolist()
NRJ_propres_normalises= sorted(valeurs_propres_H)

n=[i for i in range (1,N+1)]

plt.figure(3)
plt.plot(n,NRJ_propres_normalises)
plt.title('Representation graphique des energies normalisees')
plt.xlabel('valeurs de n')
plt.ylabel(' energies normalisees epsilon(n)')
plt.show()

# Question I.B.2.b

def epsilon_oh(R,k):
    return R*k-1/2
epsilon_oh = np.vectorize(epsilon_oh)
plt.figure(4)
plt.plot(n,NRJ_propres_normalises,'b',label='epsilon(n)')
plt.plot(n,epsilon_oh(30,n),'r',label='epsilon_oh(n)')
plt.legend()
plt.title('Representation graphique des energies normalisees du cas tudi et de l
    oscillateur harmonique')
plt.xlabel('valeurs de n')
plt.ylabel(' energies normalisees')
plt.show()

#Question I.B.2.c
## Minimisation de l'erreur quadratque

C =
    np.sum((NRJ_propres_normalises-np.array(n)**2)[30:])/len(NRJ_propres_normalises[30:])
def f_n(x):
    return x**2 + C
f=np.vectorize(f)
plt.figure(6)
plt.plot(n,NRJ_propres_normalises,'+','b',label='OH + puits infini')
plt.plot(n,epsilon_oh(30,n),'+','r',label='OH')
plt.plot(np.array(n),f(n),'y',label='ajustement quadratique')
plt.legend()
plt.title(' energies normalisees du cas tudi et de l oscillateur harmonique avec
    ajustement quadratique')
plt.xlabel('valeurs de n')
plt.ylabel(' energies normalisees')
plt.show()

# Question I.B.2.d

def n_0(R):
    k=0
    while NRJ_propres_normalises[k]<= nu_oh(R,1):
        k+=1
    return k

# Question I.B.3.a
def VP (N ,R):                          # fonction qui donnent les energies propres triees
    dans l ordre croissant et les vecteurs propres associes de l hamiltonien H(N,R)
    V= np.linalg.eig(H(N ,R))
    vp =V [0]                           #on extrait les valeurs propres
    Vp =V [1]                           #on extrait les vecteurs propres associes
    indices = np . argsort ( vp )       # indices pour trier les valeurs propres car elles
        sont dans le mauvais ordre
```

```
    vp = vp [ indices ]                 # tri des valeurs propres
    Vp = Vp [: , indices ]              #on met dans le m m e ordre les vecteurs propres
    return (vp , Vp )

X = np.linspace(0,1,200)
def Psi(n,N,R,X):                         # fonction permettant de d finir l' tat  Psi_n
    Vp = VP(N,R)[1]                      #On r cup re les coefficients des vecteurs
        propres pour pond rer les fonctions propres
    Psi = np.zeros(len( X))
    for i in range ( len (X )):
        for j in range (1,N+1) :
            Psi[i]+= Vp[j-1][n]*np.sqrt(2)*np.sin(j*np.pi*X[i])
    return Psi

def Psi0 (x,R):
    return (( np . pi *R) /(2) ) **(0.25) * np . exp (- (np.pi **2)*R*( x -1/2)**2/4)

R=30
plt.plot (X ,Psi (0 ,3 , R ,X) ,'m', label = 'N=3')
plt.plot (X , Psi (0 ,5 , R,X) ,'g', label = 'N=5')
plt.plot (X , Psi (0 ,8 , R,X) ,'y', label = 'N=8')
plt.plot (X , Psi (0 ,15 , R, X ) ,'b', label ='N =15')
plt.plot (X , Psi0 (X ,R) ,'k', label =" Courbe theorique ")
plt.legend ()
plt.xlabel ("x    =x/a")
plt.title (" Fonctions propres  d ordre  0")
plt.show ()

def Psi1(x,R):
    return (np.pi**5*R**3/(2))**(0.25)*(x-1/2)*np.exp(-np.pi**2*R*(x-1/2)**2/4)

plt.figure(8)
plt.plot (X , -Psi (1 ,3 , R ,X) ,'m', label = 'N=3')
plt.plot (X , Psi (1 ,5 , R,X) ,'g', label = 'N=5')
plt.plot (X , -Psi (1 ,8 , R,X) ,'y', label = 'N=8')
plt.plot (X , -Psi (1 ,15 , R, X ) ,'b', label ='N =15')
plt.plot (X , Psi1 (X ,R) ,'k', label =" Courbe theorique ")
plt.legend ()
plt.xlabel ("x    =x/a")
plt.title (" Fonctions propres  d ordre  1")
plt.show ()

def Psi2(x,R):
    return (np.pi*R/(8))**(0.25)*(np.pi**2*R*(x-1/2)**2-1)*np.exp(-np.pi**2*R*(x-1/2)**2/4)

plt.figure(9)
plt.plot (X , -Psi (2 ,3 , R ,X) ,'m', label = 'N=3')
plt.plot (X , -Psi (2 ,5 , R,X) ,'g', label = 'N=5')
plt.plot (X , -Psi (2 ,8 , R,X) ,'y', label = 'N=8')
plt.plot (X , -Psi (2 ,15 , R, X ) ,'b', label ='N =15')
plt.plot (X , Psi2 (X ,R) ,'k', label =" Courbe theorique ")
plt.legend ()
plt.xlabel ("x    =x/a")
plt.title (" Fonctions propres  d ordre  2")
plt.show ()
```

4.2 Scripts Partie II

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Mon Dec  8 22:25:52 2025

@author: maximerimbaud
"""

```

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

m = 9.31e-31
N = 150
a = 1e-9
b = 5*a
V0 = 0.5
hbar = 1.0546e-34
eV = 1.6e-19

def phi (n,z,b):
    return np.sqrt(1/b)*np.sin((n*np.pi/(2*b))*(z+b))

def V(z,a,v,V0):
    V = np.full_like(z,1e9)
    zone_1 = np.abs(z)<a                              # on d finit le potentiel V par zone
    zone_2 = (np.abs(z) >= a)&(np.abs(z) < b)
    V[zone_1]=-V0
    V[zone_2]=0.0
    return V

def f(z,n,m):
    return phi(n,z,b)*V(z,a,b,V0)*phi(m,z,b)  #Comme pr c demment on d finit la fonction
           int grer

def interaction_energ(n,m):
    return quad(lambda z: f(z,n,m),-b,b)[0]    #On calcule l'int raction    nergtique
        entre les  tats  n et m

def NRJ(n):
    E_joule = ((n*hbar*np.pi)**2)/(8*m*(b**2))
    E_eV = E_joule/eV
    return E_eV

def H(N):
    H = np.zeros((N,N))
    for n in range(1,N+1):
        for m in range(1,N+1):
            if n==m:
                E = NRJ(n)
            else:
                E=0
            H[n-1][m-1]= E + interaction_energ(n, m)
    return H

H = H(N)
vp, Vectp = np.linalg.eig(H)

indices = np.argsort (vp)            # indices pour trier les valeurs propres car elles
    sont dans le mauvais ordre
vp_tri = vp[indices]                 # tri des valeurs propres
Vectp_tri = Vectp[:,indices]         #on met dans le m me ordre les vecteurs propres

print("Voici les 5  nergies  propres les plus basses en eV :")
print(vp_tri[:5])


# Trac  des fonctions d'onde pour les 3  tats  li s
X = np.linspace(-b,b,500)

def psi(C,x):                    # on calcule les  tats  Psi_n    partir des coefficeints du
    vecteur propre associ
    L=np.zeros_like(x)
    for n in range(1,N+1):
        L += C[n-1]*phi(n,x,b)
    return L
```

```
73
74  # Etat 0
75  plt.figure(0)
76  C_0 = Vectp_tri[:,0]
77  psi_0 = psi(C_0,X)
78
79  norme_0 = np.sqrt(np.trapz(np.abs(psi_0)**2,X))
80  psi_0 /= norme_0
81
82  plt.plot(X*1e9, psi_0, label = " tat  0 : E = -0.4437eV")
83  plt.xlabel("z en nm")
84  plt.ylabel("Psi(z)")
85  plt.title(" tat  0")
86  plt.grid(True)
87  plt.legend()
88  plt.show()
89
90  # Etat 1
91  plt.figure(1)
92  C_1 = Vectp_tri[:,1]
93  psi_1 = psi(C_1,X)
94
95  norme_1 = np.sqrt(np.trapz(np.abs(psi_1)**2,X))
96  psi_1 /= norme_1
97
98  plt.plot(X*1e9, psi_1, label = " tat  1 : E = -0.2815eV")
99  plt.xlabel("z en nm")
100 plt.ylabel("Psi(z)")
101 plt.title(" tat  1")
102 plt.grid(True)
103 plt.legend()
104 plt.show()
105
106 # Etat 2
107 plt.figure(2)
108 C_2 = Vectp_tri[:,2]
109 psi_2 = psi(C_2,X)
110
111 norme_2 = np.sqrt(np.trapz(np.abs(psi_2)**2,X))
112 psi_2 /= norme_2
113
114 plt.plot(X*1e9, psi_2, label = " tat  2 : E = -0.0512eV")
115 plt.xlabel("z en nm")
116 plt.ylabel("Psi(z)")
117 plt.title(" tat  2")
118 plt.grid(True)
119 plt.legend()
120 plt.show()
121
122
123 Val_pot = V(X,a,b,V0)
124 Val_pot = np.minimum(Val_pot,1)
125 plt.figure(4)
126 plt.plot(X*1e9,Val_pot, 'k', lw=2,label="Potentiel")
127 plt.plot(X*1e9, vp_tri[0]+(psi_0*0.05)/30e3,lw=2, label = " tat  0 : E = -0.4437eV")
128 plt.plot(X*1e9, vp_tri[1]+(psi_1*0.05)/30e3,lw=2,label = " tat  1 : E = -0.2815eV")
129 plt.plot(X*1e9, vp_tri[2]+(psi_2*0.05)/30e3,lw=2,label = " tat  2 : E = -0.0512eV")
130 plt.xlabel("z en nm")
131 plt.ylabel(" nergie  en eV")
132 plt.title("Fonctions d'onde des  tats  li s dans le puits")
133 plt.grid(True)
134 plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))
135 plt.xlim(-b*1e9,b*1e9)
136 plt.ylim(-0.7,0.2)
137 plt.show()
```